CISS350: Data Structures and Advanced Algorithms
Quiz q10707

Name: eagraham1@cougars.ccis.edu Score:

Here's the `BTNode` class:

```
class BSTNode
{
public:
    int key_;
    BTNode * parent_;
    BTNode * left_, * right_;
};
```

For the following questions, here's an example on how to describe a binary tree in text. For this tree



you write

```
[10, [4, 25]]
[4, [1, None]]
[25, [12, 27]]
[1, [None, None]]
[12, [None, None]]
[27, [None, 29]]
[29, [None, None]]
```

Q1. In moving a pointer `p` up one node at a time during balancing, you need to keep track on the pointer that is "behind" `p` and you also need to remember the path backward by two steps, whether the 2-step path downward is a LEFT-LEFT or LEFT-RIGHT or RIGHT-LEFT or RIGHT-RIGHT. For instance if `p` is here:

```
           (10)
         /      \
     ( 4)        (25)
    /           /    \
 ( 1)    p-> (12)      (27)
                          \
                           (29)
```

and if it climbs up by one step, you have

```
           (10)
         /      \
     ( 4)        (25) <- p, down = LEFT, downdown = NONE
    /           /    \
 ( 1)     q-> (12)      (27)
                          \
                           (29)
```

and if it climbs one step up, you have

```
           (10) <- p, down = RIGHT, downdown = LEFT
         /      \
     ( 4)        (25) <- q
    /           /    \
 ( 1)        (12)      (27)
                          \
                           (29)
```

The constants `LEFT`, `RIGHT`, `NONE` are defined in the code fragment below. If `p` is the root node, after calling `climbup`, you do the obvious – for instance `p` becomes `NULL`, `q` points to the root node, `down` is `NONE`, and `downdown` is the previous value of `down`. Answer:

```
const int NONE = -1;
const int LEFT = 0;
const int RIGHT = 1;
void climbup(BTNode *& p, BTNode *& q, int & down, int & downdown)
{
  BTNode * oldp = p;

  p= p->parent_;

  q = p;

  down = downdown;

  if(p == NULL)
  {
    down = NONE;
  }
  else if(p->left_ = q)
```

```
  {
    down = LEFT;
  }
  else
  {
    down = RIGHT;
  }
}
```

Q2. For this tree:

```
           (10)
          /    \
      ( 4)      (25)
     /          /   \
 ( 1)       (12)     (27)
                        \
                         (29)
```

What is the tree after performing a right rotation at the node with key value of 10?
Answer:

```
[4, [1, 10]]
[1, [None, None]]
[10, [None, 25]]
[25, [12, 27]]
[12, [None, None]]
[27, [None, 29]]
[29, [None, None]]
```

Q3. For this BST:

```
           (10)
          /    \
      ( 4)      (25)
     /          /   \
 ( 1)       (12)     (27)
                        \
                         (29)
```

insert 30 and then balance it (if necessary). Write the resulting tree in text below.
Answer:

```
[25, [10, 29]]
[10, [4, 12]]
[29, [27, 30]]
[4, [1, None]]
[12, [None, None]]
[27, [None, None]]
[30, [None, None]]
[1, [None, None]]
```

Q4. For this BST:



insert 2 and then balance it (if necessary). Write the resulting tree in text below. Answer:

```
[10, [4, 25]]
[2, [1, 4]]
[25, [12, 27]]
[1, [None, None]]
[4, [None, None]]
[12, [None, None]]
[27, [None, 29]]
[29, [None, None]]
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.